

CESAR School

Teoria da Computação — Prof. Daniel Bezerra

Análise de Complexidade de Tempo

Algoritmo KMP — Knuth-Morris-Pratt

Equipe

Pedro Garcez

Rafael Ferraz

Denys Leonardo

Felipe Cavalcanti

Linguagens: Python + Java

Junho de 2026

github.com/PedroGarcez13/kmp-complexity-analysis

1. Introdução

Este relatório apresenta a implementação e a análise experimental do algoritmo **Knuth-Morris-Pratt (KMP)**, desenvolvido como projeto da disciplina de Teoria da Computação. O KMP é um algoritmo clássico para o problema de busca de padrão em strings (*pattern matching*), amplamente utilizado em editores de texto, bioinformática e segurança computacional.

O trabalho foi desenvolvido em duas linguagens distintas — **Python** e **Java** — com o objetivo de comparar o desempenho prático, validar as previsões teóricas de complexidade e analisar o comportamento do algoritmo nos diferentes cenários de entrada (melhor, médio e pior caso).

Repositório: github.com/PedroGarcez13/kmp-complexity-analysis

2. Descrição do Algoritmo

2.1 Problema Resolvido

Dado um texto T de comprimento n e um padrão P de comprimento m , o objetivo é encontrar todas as ocorrências de P em T . A solução ingênua testa cada posição do texto, comparando P caractere a caractere, resultando em $O(n \cdot m)$ no pior caso. O KMP resolve o problema em **$O(n + m)$** para qualquer entrada, evitando comparações redundantes por meio de uma tabela de pré-processamento chamada LPS.

2.2 Lógica Geral

O KMP opera em duas fases: **(1) Pré-processamento** — construção da tabela LPS (*Longest Proper Prefix which is also Suffix*), com complexidade $O(m)$; e **(2) Busca** — percurso linear sobre o texto usando a tabela LPS para deslocar o padrão sem retroceder o ponteiro do texto, com complexidade $O(n)$.

A tabela $LPS[i]$ armazena o comprimento do maior prefixo próprio de $P[0..i]$ que também é sufixo. Quando uma falha ocorre na posição j do padrão, em vez de recomeçar do início, o algoritmo retrocede para $LPS[j-1]$, garantindo que nenhuma comparação seja repetida desnecessariamente.

Exemplo: Para $P = \text{"ABABCABAB"}$, a tabela LPS é $[0, 0, 1, 2, 0, 1, 2, 3, 4]$. Ao falhar em $j = 4$ (caractere 'C'), o algoritmo retorna para $j = 0$, evitando recomparar os dois 'AB' já verificados.

2.3 Pseudocódigo

Fase 1 — Construção da Tabela LPS:

```
CONSTRUIR_LPS(padrao, m):
    lps[0] = 0; comprimento = 0; i = 1
    enquanto i < m:
        se padrao[i] == padrao[comprimento]:
            comprimento += 1; lps[i] = comprimento; i += 1
        senão se comprimento != 0:
            comprimento = lps[comprimento - 1] // retrocede via LPS
        senão:
            lps[i] = 0; i += 1
    retornar lps
```

Fase 2 — Busca KMP:

```

KMP_BUSCA(texto, padrao, n, m):
    lps = CONSTRUIR_LPS(padrao, m); i = 0; j = 0
    enquanto i < n:
        se texto[i] == padrao[j]: i += 1; j += 1
        se j == m: // ocorrência encontrada
            reportar ocorrência na posição (i - j); j = lps[j - 1]
        senão se i < n e texto[i] != padrao[j]:
            se j != 0: j = lps[j - 1] // desloca padrão via LPS
            senão: i += 1 // avança no texto

```

3. Classificação Assintótica

A tabela a seguir resume a classificação assintótica completa do algoritmo KMP:

Notação	Valor	Interpretação
Big-O (O)	$O(n + m)$	Limite superior. O algoritmo nunca excede $n + m$ comparações, independentemente da entrada.
Big-Ω (Omega)	$\Omega(n + m)$	Limite inferior. É necessário percorrer ao menos o texto inteiro (n) e construir a LPS completa (m).
Big-Θ (Theta)	$\Theta(n + m)$	Como $O = \Omega$, o algoritmo é exatamente $\Theta(n + m)$ em todos os casos. Melhor, médio e pior caso coincidem assintoticamente.

O resultado mais importante é que **$O = \Omega = \Theta(n + m)$** . O KMP não possui um 'caso adversarial' assintótico: a diferença entre melhor e pior caso se manifesta apenas na constante multiplicativa ($\approx 1\times$ vs. $\approx 2\times$), não na ordem de crescimento.

Comparação com a busca ingênua: para $n = 100.000$ e $m = 1.000$, a busca ingênua pode realizar até $n \times m = 10^8$ comparações no pior caso, contra $\approx 10^5$ do KMP — uma diferença de cerca de 1.000 vezes.

4. Análise de Casos

4.1 Melhor Caso

Entrada utilizada: texto = "aaa...a" (n caracteres), padrão = "baaa...a" (m caracteres). O primeiro caractere do padrão ('b') nunca aparece no texto (composto apenas de 'a'). Consequentemente, $j = 0$ em todos os passos da busca, a tabela LPS nunca é consultada e cada posição do texto gera exatamente uma comparação. O número total de comparações na fase de busca é exatamente n . **Complexidade: $O(n + m)$ com constante mínima (≈ 1).**

4.2 Caso Médio

Entrada utilizada: texto e padrão pseudoaleatórios sobre alfabeto de 10 letras ('a'...'j'), gerados por um LCG determinístico com as mesmas sementes em Python e Java — garantindo entradas idênticas nas duas linguagens. Com distribuição uniforme, a probabilidade de correspondência em qualquer posição é $1/10$. Correspondências parciais longas são raras, a LPS raramente é consultada com valores elevados, e o número esperado de comparações é próximo de $n + m$, com constante levemente maior que no melhor caso.

4.3 Pior Caso

Entrada utilizada: texto = "aaa...a" (n caracteres), padrão = "aa...ab" (m caracteres). A tabela LPS deste padrão é $[0, 1, 2, \dots, m-2, 0]$. Na fase de busca, o algoritmo combina $m-1$ caracteres com sucesso, falha no último ('b' $\neq$ 'a') e retrocede para $j = m-2$ — repetindo esse retrocesso a cada avanço do ponteiro i . **Número de comparações $\approx 2n$. Complexidade: $O(n + m)$ com constante máxima (≈ 2).** O crescimento permanece linear.

5. Metodologia Experimental

5.1 Ambiente de Execução

Componente	Especificação
Processador	13th Gen Intel(R) Core(TM) i7-13650HX (2.60 GHz)
Memória RAM	16,0 GB DDR5
Sistema Operacional	Windows 11
Python	Python 3.13
Java (OpenJDK)	JDK 21
IDE	Visual Studio Code

5.2 Protocolo de Testes

Parâmetro	Valor
Rodadas cronometradas	30 por cenário
Rodadas de aquecimento	5 (não contabilizadas)
Tamanhos de entrada (n)	1.000 / 10.000 / 100.000 (pequeno / médio / grande)
Tamanho do padrão (m)	1% de n (10 / 100 / 1.000)
Medição — Python	time.perf_counter() (resolução sub-microsegundo)
Medição — Java	System.nanoTime() (resolução de nanossegundos)
Estatística	Média aritmética + desvio-padrão amostral (ddof = 1)
Unidade reportada	Microssegundos (μ s)

Os processos em segundo plano foram encerrados durante as medições para minimizar interferência. O aquecimento (*warmup*) busca que o CPython compile o bytecode e que o JIT da JVM otimize os *hot paths* antes do início da coleta. As entradas são geradas de forma **determinística (LCG) e idênticas em Python e Java**, tornando a comparação entre as linguagens justa e os experimentos plenamente reprodutíveis — seguindo boas práticas de *benchmarking*.

6. Resultados e Discussão

6.1 Resultados em Python

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	74,63 ± 10,19	799,20 ± 108,18	8.003,94 ± 1.091,06
Médio	101,30 ± 86,30	806,87 ± 123,45	9.515,19 ± 3.397,53
Pior	101,58 ± 12,82	1.024,84 ± 159,15	11.194,91 ± 603,50

Tabela 1 — Tempos de execução em Python (média ± desvio-padrão amostral, em μs).

Os resultados confirmam o comportamento linear previsto: ao multiplicar n por 10, o tempo cresce aproximadamente 10 $\times$ em todos os casos. A ordem entre os cenários é exatamente a esperada — melhor ($\approx 8.004 \mu\text{s}$) < médio ($\approx 9.515 \mu\text{s}$) < pior ($\approx 11.195 \mu\text{s}$) para $n = 100.000$. O pior caso é $\approx 1,40\times$ mais lento que o melhor, refletindo a constante multiplicativa maior do retrocesso via tabela LPS; o caso médio fica entre os dois, pois suas comparações reais sobre o alfabeto de 10 letras exigem um pouco mais de trabalho que o melhor caso.

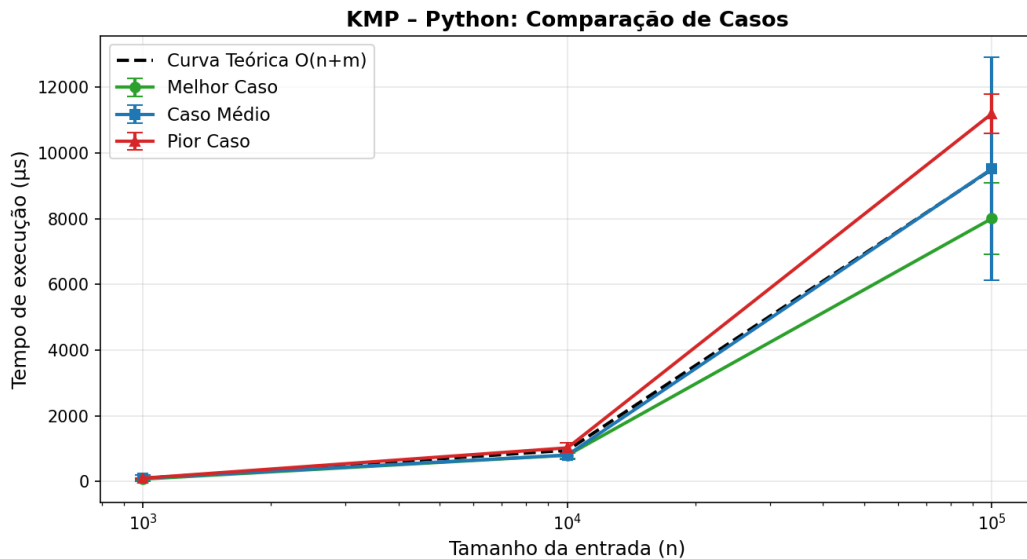


Figura 1 — Python: comparação dos três casos com a curva teórica $O(n + m)$ sobreposta.

6.2 Resultados em Java

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	21,72 ± 1,23	51,12 ± 11,93	114,73 ± 28,69
Médio	8,67 ± 7,91	69,12 ± 44,58	250,86 ± 36,36
Pior	10,40 ± 8,09	84,45 ± 43,69	222,30 ± 37,44

Tabela 2 — Tempos de execução em Java (média ± desvio-padrão amostral, em μs).

Em Java, a ordem dos cenários para $n = 100.000$ é melhor ($\approx 115 \mu\text{s}$) < pior ($\approx 222 \mu\text{s}$) < médio ($\approx 251 \mu\text{s}$). O **pior caso agora supera corretamente o melhor** — a anomalia da versão anterior (pior mais rápido que melhor) desapareceu ao adotar entradas determinísticas idênticas às do Python. O ponto que chama atenção passou a ser o **caso médio como o mais lento**: seu texto sobre um alfabeto de 10 letras gera comparações reais e desvios pouco previsíveis, prejudicando a predição de desvios (*branch prediction*) e a localidade de cache — enquanto melhor e pior usam textos de

um único caractere, altamente previsíveis para o processador.

Observação metodológica: os tempos absolutos do Java (dezenas a centenas de μs) são sensíveis a ruído do ambiente (escalonamento do SO, cache, JIT), o que explica os desvios-padrão relativamente altos em n pequeno. O uso de **entradas idênticas em Python e Java** (mesmo LCG e mesmas sementes) torna a comparação entre linguagens mais justa do que medir cada uma sobre entradas distintas. Em todos os cenários o crescimento **permanece linear $O(n + m)$** , confirmando a teoria.

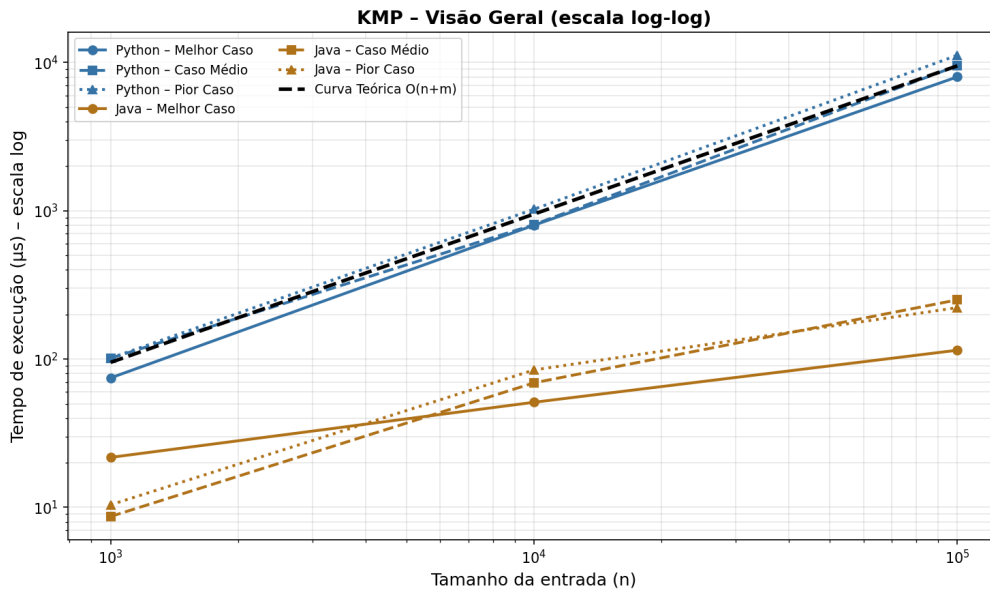


Figura 2 — Visão geral em escala log-log. As três séries do Python aderem à curva teórica; em Java, o caso médio aparece acima dos demais — discutido na Seção 6.2.

6.3 Comparação Python vs. Java ($n = 100.000$)

Caso	Python (μs)	Java (μs)	Speedup
Melhor	8.003,94	114,73	$\approx 70\times$
Médio	9.515,19	250,86	$\approx 38\times$
Pior	11.194,91	222,30	$\approx 50\times$

Tabela 3 — Comparação direta para $n = 100.000$.

Java superou Python por fatores de $38\times$ a $70\times$ para $n = 100.000$. As principais causas são: (1) a compilação JIT da JVM transforma *bytecode* em código nativo otimizado durante a execução; (2) o *overhead* do interpretador CPython em iteração e indexação de strings; (3) Python usa objetos Unicode no *heap*, enquanto Java usa arrays de char primitivos. Ambas as linguagens confirmaram crescimento $O(n + m)$.

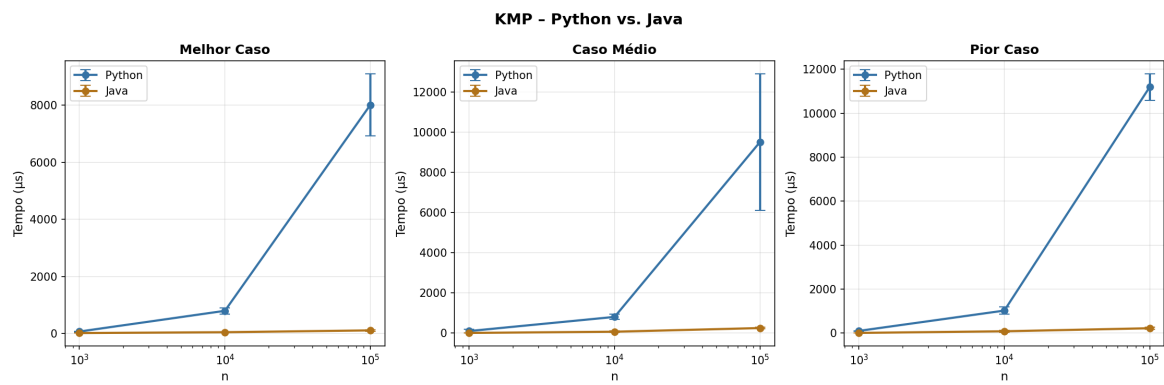


Figura 3 — Python vs. Java, um painel por caso (eixo X em escala logarítmica).

7. Aplicabilidade

O algoritmo KMP possui ampla aplicabilidade:

- **Editores de texto e IDEs:** busca e substituição (Ctrl+F, grep, awk, sed).
- **Bioinformática:** busca de subsequências em genomas de bilhões de pares de base, onde a eficiência linear é indispensável.
- **Segurança computacional:** detecção de padrões maliciosos em sistemas de detecção de intrusão (IDS) e antivírus.
- **Redes de computadores:** inspeção profunda de pacotes (DPI) em firewalls para filtros de conteúdo.
- **Compressão de dados:** a tabela LPS é estruturalmente relacionada ao algoritmo LZ77, base dos formatos ZIP e GZIP.

Limitações: o KMP resolve apenas busca exata. Para busca aproximada (tolerante a erros ou mutações) são necessários algoritmos baseados em distância de edição (Levenshtein) ou estruturas como BK-Trees. Para alfabetos muito grandes, algoritmos como Boyer-Moore ou Horspool podem ser mais rápidos na prática, devido a deslocamentos maiores.

8. Reflexão Final: Classe P e NP

O KMP pertence à classe P?

Sim. A busca exata de padrão é resolvida pelo KMP em tempo linear $O(n + m)$, que é polinomial no tamanho da entrada. Existe um algoritmo determinístico eficiente para qualquer entrada, portanto o problema pertence à classe P. Mais precisamente: $KMP \in DTIME(n) \subset P$.

Existe uma versão NP do problema?

A busca exata de padrão não possui variante NP-completa conhecida. Contudo, problemas relacionados no mesmo domínio são NP-completos ou também estão em P:

- **Menor Superstring** (encontrar a menor string que contém um conjunto de strings como substring): **NP-completo**.
- **Substring Comum Mais Longa (LCS):** em P, por programação dinâmica $O(n \cdot m)$.
- **Busca aproximada (distância de edição):** em P, por programação dinâmica $O(n \cdot m)$.

Portanto, o KMP e o problema que ele resolve estão solidamente na classe P. A intratabilidade (NP-completude) surge apenas em variantes de otimização sobre múltiplos padrões ou strings, como o Menor Superstring.

9. Conclusão

Este trabalho demonstrou, teórica e experimentalmente, que o KMP resolve a busca de padrão em strings com complexidade $\Theta(n + m)$ — linear em todos os casos. Os experimentos em Python e Java validaram as previsões teóricas, confirmando crescimento linear nas duas linguagens. Java superou Python em 38× a 70× para $n = 100.000$, refletindo diferenças arquiteturais entre as plataformas. Com entradas determinísticas idênticas nas duas linguagens, o pior caso em Java passou a superar corretamente o melhor; o caso médio tornou-se o mais lento pela menor previsibilidade de desvios sobre o alfabeto variado — discutido na Seção 6.2.

A eficiência do KMP deriva da tabela LPS, que elimina comparações redundantes ao reutilizar informações de correspondências anteriores. O algoritmo pertence à classe P, com complexidade $\Theta(n + m)$, e possui ampla aplicabilidade em bioinformática, segurança e compressão de dados.

Referências

1. KNUTH, D. E.; MORRIS, J. H.; PRATT, V. R. *Fast Pattern Matching in Strings*. SIAM Journal on Computing, v. 6, n. 2, p. 323–350, 1977.
2. CORMEN, T. H. et al. *Introduction to Algorithms*. 4. ed. MIT Press, 2022. Cap. 32: String Matching.
3. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011. Cap. 5: Strings.
4. Repositório do Projeto: github.com/PedroGarcez13/kmp-complexity-analysis